Nama : Muchammad Fakhri Firdaus
Kelas : TI-1G
No. Presensi : 12

Percobaan 1

Kode Program:

```java
import java.util.Scanner;
public class Star12 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan nilai n: ");
        int n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            System.out.print("*");
        }
        System.out.println();
        sc.close();
    }
}
```

Output:

```
Masukkan nilai n: 3
***
```

```
Masukkan nilai n: 7
*******
```

1. **Output inisialisasi i = 1 diubah menjadi i = 0 :**

```
Masukkan nilai n: 5
******
```

Meskipun nilai awal i dimulai dari 0 (i = 0), kondisi perulangan yang digunakan adalah *i* <= *n* dengan n = 5. Perulangan akan terus berjalan selama nilai i memenuhi nilai 0, 1, 2, 3, 4, 5. Jika dihitung total iterasi dari 0 sampai 5, perulangan akan berjalan sebanyak **6 kali** (yaitu saat $i = 0,1,2,3,4,5$). Namun, karena perintah di dalam perulangan adalah *System.out.print("*");* yang mengeksekusi cetak karakter pada setiap iterasi tanpa batasan indeks khusus selain jumlah putaran, maka total karakter * yang keluar akan berjumlah **6 buah.**

2. **Output kondisi i <= n diganti menjadi i < n :**

```
Masukkan nilai n: 5
****
```

Hasilnya berbeda karena penggunaan operator perbandingan < (kurang dari) tidak menyertakan nilai n itu sendiri ke dalam batas eksekusi. Jika i = 1 dan n = 5, maka perulangan hanya akan berjalan untuk nilai i = 1, 2, 3, 4 (total 4 kali jika dimulai dari 1, atau 5 kali jika dimulai dari 0).

3. **Prediksi Output:**

```
Masukkan nilai n: 9
2 4 6 8
```

Modifikasi (Bilangan Ganjil Positif): Agar kode tersebut dapat mencetak bilangan **ganjil** positif yang tidak melebihi n, kita perlu mengubah nilai awal inisialisasi i menjadi 1 dan mengubah pola increment-nya tetap melompat sebanyak 2 (i += 2).

Modifikasi kodenya menjadi:

```java
for (int i = 1; i <= n; i += 2) {
    System.out.print(i + " ");
}
```

4. Perubahan dari i++ (penambahan nilai i) menjadi i-- (pengurangan nilai i) akan menyebabkan infinite loop (perulangan tak terbatas) karena logika pergerakan nilai variabel kontrol berlawanan arah dengan kondisi pemberhentian:
    1. Dengan asumsi n bernilai positif (misalnya n = 5) dan inisialisasi awal i = 1, kondisi awal 1 <= 5 bernilai true.
    2. Karena menggunakan i--, pada setiap iterasi nilai i bukannya bertambah mendekati n, melainkan terus berkurang menjauhi n (misalnya: 1, lalu 0, -1, -2, -3, dan seterusnya menjadi angka negatif).
    3. Akibatnya, kondisi perulangan i <= n akan selamanya bernilai true karena angka negatif akan selalu lebih kecil daripada angka positif n, sehingga program tidak akan pernah menemukan titik di mana perulangan harus berhenti secara alami.

Percobaan 2

Kode Program:

```java
import java.util.Scanner;
You, 6 minutes ago | author (You)
public class Rectangle12 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);

        System.out.print("Masukkan jumlah baris: ");
        int baris = sc.nextInt();

        System.out.print("Masukkan jumlah kolom: ");
        int kolom = sc.nextInt();

        for (int i = 1; i <= baris; i++) {
            for (int j = 1; j <= kolom; j++) {
                System.out.print("*");
            }
            System.out.println();
        }
        sc.close();

    }

}
```

Output (4,2):

```
Masukkan jumlah baris: 4
Masukkan jumlah kolom: 2
**
**
**
**
```

Output (1,5):

```
Masukkan jumlah baris: 1
Masukkan jumlah kolom: 5
*****
```

1. Fungsi variabel i dan j pada program:
    - **Variabel i:** Berfungsi sebagai penghitung/kontrol iterasi untuk **outer loop** (perulangan luar), yang merepresentasikan jumlah baris yang akan dicetak secara vertikal ke bawah dari baris ke-1 hingga baris.
    - **Variabel j:** Berfungsi sebagai penghitung/kontrol iterasi untuk **inner loop** (perulangan dalam), yang merepresentasikan jumlah kolom atau karakter bintang (*) yang dicetak secara horizontal ke samping pada setiap barisnya, dari kolom ke-1 hingga kolom.

2. Perintah (4,7) :

```
Masukkan jumlah baris: 4
Masukkan jumlah kolom: 7
*******
*******
*******
*******
```

- **Outer loop:** Dijalankan sebanyak **4 kali**.
- **Inner loop:** Dijalankan secara total sebanyak 4 x 7 = **28 kali**.
- **Perintah** *print("*"):* Dijalankan sebanyak **28 kali**

3. Output:

```
Rectangle12
Masukkan jumlah baris: 3
Masukkan jumlah kolom: 6
***
***
***
***
***
***
```

Bentuk output pola yang dicetak di layar akan bergeser dimensinya, dari yang awalnya membentuk persegi panjang dengan 3 baris ke bawah dan 6 kolom ke samping, menjadi **6 baris ke bawah dan 3 kolom ke samping**. Secara total karakter yang dicetak tetap sama ($6 \times 3 = 18$ karakter), tetapi orientasi matriks/tampilannya bertukar dari orientasi awal.

4. Seluruh karakter bintang (*) dari setiap baris akan tercetak **menyambung menjadi satu baris horizontal panjang tanpa ada pemisah baris (enter)**. Perintah System.out.println() di luar inner loop berfungsi sebagai pemindah baris (ganti baris baru) setiap kali sebuah baris selesai dicetak sepenuhnya oleh inner loop. Jika dihapus, bentuk persegi panjang atau matriks dua dimensi akan rusak dan berantakan menjadi satu baris panjang.
5. Modifikasi Program:

```java
import java.util.Scanner;

public class Rectangle12 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner (System.in);

        System.out.print("Masukkan jumlah baris: ");
        int baris = sc.nextInt();

        System.out.print("Masukkan jumlah kolom: ");
        int kolom = sc.nextInt();

        for (int i = 1; i <= baris; i++) {
            for (int j = 1; j <= kolom; j++) {
                System.out.print("#");
            }
            System.out.println();
        }
        sc.close();

    }

}
```

Percobaan 3

Kode Soal Langkah 4:

```java
import java.util.Scanner;
You, 54 minutes ago | author (You)
public class Triangle12 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan nilai n: ");
        int n = sc.nextInt();

        int i = 0;
        while (i <= n) {
            System.out.println("[i=" + i + "]");
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            i++;
        }
        sc.close();
    }
}
```

Output:

```
Masukkan nilai n: 5
[i=0]
[i=1]
*[i=2]
**[i=3]
***[i=4]
****[i=5]
*****
```

5. Jalankan n=5

a. **Outer loop** dijalankan sebanyak **6 kali** (yaitu saat nilai $i = 0,1,2,3,4,5$). **Tidak sama dengan jumlah baris yang diinginkan**. Program ini mencetak total 6 baris (karena dimulai dari $i = 0$), padahal nilai $n = 5$ biasanya dimaksudkan untuk 5 baris saja. Selain itu, pada iterasi pertama saat $i = 0$, *inner loop* tidak akan berjalan sama sekali (karena $j < i$ yaitu $0 < 0$ bernilai *false*), sehingga hanya mencetak baris teks [i=0] tanpa ada karakter bintang (*).

b. Saat $i = 0$, jumlah karakter * yang tercetak adalah **0 (tidak ada bintang yang tercetak)**. Ketika $i = 0$, program mengeksekusi System.out.println("[i=0]");, lalu masuk ke *inner loop* dengan kondisi while (j < i) atau while (0 < 0). Karena kondisi tersebut bernilai *false*, *inner loop* langsung dilewati dan tidak ada perintah System.out.print("*") yang dijalankan.

c. Pada kode ini, karakter bintang **tidak tampil dalam satu baris**, melainkan tercetak membentuk pola segitiga bertingkat ke bawah. Hal ini dikarenakan adanya perintah System.out.println("[i=" + i + "]"); di dalam *outer loop* yang berfungsi memindahkan kursor ke baris baru (enter) setiap kali perpindahan nilai i, serta penggunaan perintah System.out.print("*") di dalam *inner loop* untuk mencetak bintang secara horizontal sebelum akhirnya baris baru terbentuk.

Nama : Muchammad Fakhri Firdaus
Kelas : TI-1G
No. Presensi : 12

Kode Program:

```java
import java.util.Scanner;
You, 1 hour ago | author (You)
public class Triangle12 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan nilai n: ");
        int n = sc.nextInt();

        int i = 1;
        while (i <= n) {
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i++;
        }
        i = n;
        while(i >= 1){
            int j = 0;
            while (j < i) {
                System.out.print("*");
                j++;
            }
            System.out.println();
            i--;
        }
        sc.close();
    }
}
You, 1 hour ago • PB3 …
```

Output:

```
Masukkan nilai n: 5
*
**
***
****
*****
*****
****
***
**
*
```

1. **Ketidaksesuaian Output Awal:** Pada kode saat ini terdapat dua blok perulangan while secara berurutan. Blok pertama mencetak segitiga siku-siku bertambah (1 hingga n), lalu blok kedua mencetak segitiga siku-siku berkurang (n hingga 1). Akibatnya, output yang dihasilkan membentuk gabungan dua segitiga (belah ketupat/dua puncak bertumpuk dengan puncak terbesar n tercetak 2 kali berturut-turut), bukan sekadar **segitiga bertingkat tunggal**.
   Dua Perbaikan yang Diperlukan:
   - **Menghapus/menonaktifkan blok while kedua** (i = n; while(i >= 1)...) agar hanya perulangan pertama saja yang dieksekusi.
   - **Mengatur pencetakan System.out.println() berada di posisi yang tepat** setelah *inner loop* selesai pada setiap baris untuk memastikan pemisahan baris terjadi dengan benar.

2. **Peran Variabel i dan j:**
   - **Variabel i:** Berfungsi sebagai indikator/penghitung jumlah **baris** pada *outer loop*. Nilai i juga menentukan berapa banyak jumlah bintang yang harus dicetak pada baris tersebut.
   - **Variabel j:** Berfungsi sebagai indikator/penghitung jumlah **kolom** (bintang) pada *inner loop* untuk mencetak karakter * secara horizontal ke samping pada baris yang aktif.

   Mengapa j Harus Di-inisialisasi Kembali ke 0 Setiap Iterasi *Outer Loop*?
   Nilai j harus di-reset ke 0 (int j = 0;) setiap kali *outer loop* berpindah ke baris baru agar proses perhitungan bintang pada baris berikutnya selalu dimulai dari awal (kolom ke-0).
   Apa yang Terjadi Jika j Tidak Di-reset?
   Jika j tidak di-reset, nilai j akan tetap menyimpan nilai terakhir dari iterasi baris sebelumnya (di mana j sudah bernilai sama dengan i). Akibatnya, pada baris-baris berikutnya kondisi j < i langsung bernilai **false**, sehingga *inner loop* tidak akan pernah berjalan lagi dan bintang hanya akan tercetak di baris pertama saja.
3. Untuk mengubah pola segitiga siku-siku biasa menjadi **segitiga terbalik** (dimulai dari n bintang di baris pertama turun hingga 1 bintang di baris terakhir), perubahannya difokuskan pada tiga bagian *outer loop* berikut:
   - **Inisialisasi (Nilai Awal):** Ubah nilai awal i dari 1 menjadi n (i = n).
   - **Kondisi (Batas Perulangan):** Ubah kondisi syarat perulangan dari i <= n menjadi i >= 1
   - **Perubahan Nilai (*Update*):** Ubah operasi *increment* i++ menjadi *decrement* i--

Percobaan 4

Kode program:

```java
import java.util.Scanner;
You, 1 hour ago | author (You)
public class NilaiKelompok12 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan jumlah kelompok : ");
        int jumlahKelompok = sc.nextInt();
        System.out.print("Masukkan jumlah penilai : ");
        int jumlahPenilai = sc.nextInt();

        if (jumlahKelompok <= 0 || jumlahPenilai <= 0) {
            System.out.println("Jumlah kelompok dan penilai harus > 0.");
        } else {
            int nomorKelompok = 1;
            double rataTertinggi = -1;
            int kelompokTerbaik = 0;

            while (nomorKelompok <= jumlahKelompok) {
                System.out.println("Kelompok " + nomorKelompok);
                double totalNilai = 0;

                for (int j = 1; j <= jumlahPenilai; j++) {
                    int nilai;
                    do {
                        System.out.print("Nilai dari Penilai " + j + ": ");
                        nilai = sc.nextInt();
                        if (nilai < 0 || nilai > 100) {
                            System.out.println("Nilai harus 0-100. Ulangi input!");
                        }
                    } while (nilai < 0 || nilai > 100);
                    totalNilai += nilai;
                }

                double rataNilai = totalNilai / jumlahPenilai;
                System.out.println("Kelompok " + nomorKelompok + ": nilai rata-rata = " + rataNilai);

                if (rataNilai > rataTertinggi) {
                    rataTertinggi = rataNilai;
                    kelompokTerbaik = nomorKelompok;
                }
                nomorKelompok++;
            }

            System.out.println("=======================================");
            System.out.println("Kelompok terbaik    : " + kelompokTerbaik);
            System.out.println("Rata-rata tertinggi : " + rataTertinggi);
        }
        sc.close();
    }
}
```

1. totalNilai digunakan untuk menghitung akumulasi penjumlahan nilai dari para penilai **hanya untuk kelompok yang sedang diproses saat itu**. Jika tidak di-*reset* menjadi 0 di awal *outer loop*, nilai dari kelompok sebelumnya akan terbawa dan terakumulasi ke kelompok berikutnya, sehingga perhitungan rata-rata kelompok selanjutnya menjadi tidak akurat. Sedangkan, **rataTertinggi tidak boleh di-*reset*** karena berfungsi menyimpan nilai rata-rata maksimum dari **seluruh kelompok yang telah dibandingkan** sejauh ini. Jika di-*reset* di dalam *outer loop* pada setiap kelompok, variabel ini akan kehilangan ingatan nilai tertinggi dari kelompok-kelompok sebelumnya.
2. Badan perulangan FOR dijalankan sebanyak **6 kali**. Perulangan luar *(while)* berjalan sebanyak jumlah kelompok, yaitu **3 kali**. Untuk setiap 1 kelompok, perulangan for berjalan sebanyak jumlah penilai, yaitu **2 kali**. Total eksekusi badan perulangan for secara keseluruhan = $3 \times 2 = 6$ kali.
3. Akan terjadi **pembagian bulat (*integer division*)**, di mana bagian desimal (pecahan) dari hasil pembagian akan **terpotong/dibuang (*truncated*)**, bukan dibulatkan.
   Contoh Nilai: Jika totalNilai = 17 (tipe int) dan jumlahPenilai = 2 (tipe int), maka operasi totalNilai / jumlahPenilai (17/2) akan menghasilkan 8, bukan 8.5.

4. Pemberian nilai awal -1 memastikan bahwa nilai rata-rata kelompok mana pun (termasuk yang bernilai 0) akan tetap **lebih besar** daripada nilai awal *rataTertinggi,* sehingga kelompok pertama akan selalu tercatat sebagai perbanding awal yang valid.
   Contoh:
   Misalkan ada 2 kelompok, dan seluruh penilai memberikan nilai 0 untuk semua kelompok (semua input nilai = 0). Nilai rataNilai Kelompok 1 = 0.0. Saat evaluasi if (rataNilai > rataTertinggi) → if (0.0 > 0.0) bernilai **false**. Akibatnya, kelompokTerbaik tidak akan pernah ter-update dan tetap bernilai 0 (kelompok tidak terdeteksi).
5. **Jenis Perulangan:**
   - **Perulangan Tingkat 1 — while (Outer Loop):**
     Mengontrol perulangan antar-kelompok dari kelompok ke-1 hingga mencapai jumlahKelompok.
   - **Perulangan Tingkat 2 — for (Middle Loop):**
     Mengontrol iterasi pencatatan nilai dari setiap penilai (dari penilai ke-1 hingga jumlahPenilai) untuk kelompok yang sedang aktif.
   - **Perulangan Tingkat 3 — do-while (Inner Loop):**
     Melakukan **validasi input nilai**. Perulangan ini memastikan input nilai penilai berada pada rentang yang valid ($0 - 100$), dan akan memaksa pengguna mengulang input jika nilai yang dimasukkan di luar rentang tersebut.

Tugas 1: Pola Bilangan Bertingkat
Kode Program:

```java
import java.util.Scanner;
public class Tugas1_12 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n;
        int total = 0;

        System.out.print("Masukkan nilai n: ");
        n = sc.nextInt();

        for (int i = 1; i <= n; i++) {
            for (int j = 1; j <= i; j++) {
                System.out.print(j + " ");
                total++;
            }
            System.out.println();
        }
        System.out.println("Total angka tercetak: " + total);

        sc.close();
    }
}
```

Contoh Output:

```
Masukkan nilai n: 5
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
Total angka tercetak: 15
```

Nama : Muchammad Fakhri Firdaus
Kelas : TI-1G
No. Presensi : 12

Tugas 2: Tabel Perkalian

Kode Program:

```java
import java.util.Scanner;

public class Tugas2_12 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Masukkan nilai n: ");
        int n = sc.nextInt();

        if (n < 3) {
            System.out.println("Nilai n minimal adalah 3");
        } else {
            int jumlah = 0;
            for (int i = 1; i <= n; i++) {
                for (int j = 1; j <= n; j++) {
                    int hasil = i * j;
                    jumlah += hasil;
                    System.out.printf("%4d", hasil);
                }
                System.out.println();
            }
            System.out.println("Jumlah seluruh angka: " + jumlah);
        }
        sc.close();
    }
}
```

Contoh Output:

```
Masukkan nilai n: 4
   1   2   3   4
   2   4   6   8
   3   6   9  12
   4   8  12  16
Jumlah seluruh angka: 100
```

Nama : Muchammad Fakhri Firdaus
Kelas : TI-1G
No. Presensi : 12

Tugas 3: Rekap Penjualan Beberapa Toko

```java
import java.util.Scanner;
You, 2 hours ago | author (You)
public class Tugas3_12 {
    Run | Debug
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int jumlahToko = 0;

        while (true) {
            System.out.print("Masukkan jumlah toko (minimal 1): ");
            jumlahToko = sc.nextInt();

            if (jumlahToko ≥ 1) {
                break;
            }
            System.out.println("Jumlah toko minimal 1. Silakan masukkan ulang.");
        }

        int totalBarang = 0;
        int maxBarang = -1;
        int tokoTerbanyak = 0;

        for (int i = 1; i ≤ jumlahToko; i++) {
            int jumlahPelanggan = 0;
            while (true) {
                System.out.print("Masukkan jumlah pelanggan untuk Toko " + i + " (tidak boleh negatif): ");
                jumlahPelanggan = sc.nextInt();
                if (jumlahPelanggan ≥ 0) {
                    break;
                }
                System.out.println("Jumlah pelanggan tidak boleh negatif.");
            }

            int totalBarangToko = 0;
            for (int j = 1; j ≤ jumlahPelanggan; j++) {
                int jumlahBarang = 0;
                while (true) {
                    System.out.print(" - Masukkan jumlah barang yang dibeli pelanggan ke-" + j + " (tidak boleh negatif): ");
                    jumlahBarang = sc.nextInt();
                    if (jumlahBarang ≥ 0) {
                        break;
                    }
                    System.out.println(" - Jumlah barang tidak boleh negatif.");
                }
                totalBarangToko += jumlahBarang;
            }
            totalBarang += totalBarangToko;
            System.out.println("Toko" + i + " - pelanggan: " + jumlahPelanggan + ", total barang: " + totalBarangToko);

            if (totalBarangToko > maxBarang) {
                maxBarang = totalBarangToko;
                tokoTerbanyak = i;
            }
            System.out.println();
        }
        System.out.println("Total barang seluruh toko: " + totalBarang);
        System.out.println("Toko dengan barang terbanyak: " + tokoTerbanyak);

        sc.close();
    }
}
```

Contoh Output:

```
Masukkan jumlah toko (minimal 1): 2
Masukkan jumlah pelanggan untuk Toko 1 (tidak boleh negatif): 2
 - Masukkan jumlah barang yang dibeli pelanggan ke-1 (tidak boleh negatif): 3
 - Masukkan jumlah barang yang dibeli pelanggan ke-2 (tidak boleh negatif): 2
Toko1 - pelanggan: 2, total barang: 5

Masukkan jumlah pelanggan untuk Toko 2 (tidak boleh negatif): 3
 - Masukkan jumlah barang yang dibeli pelanggan ke-1 (tidak boleh negatif): 1
 - Masukkan jumlah barang yang dibeli pelanggan ke-2 (tidak boleh negatif): 4
 - Masukkan jumlah barang yang dibeli pelanggan ke-3 (tidak boleh negatif): 2
Toko2 - pelanggan: 3, total barang: 7

Total barang seluruh toko: 12
Toko dengan barang terbanyak: 2
```